
Claude Certified Developer 202X

Contents

Bird's-Eye Skinny	3
"USB-C port for AI" — Model Context Protocol (MCP).....	5
Couple of clarifying Q's:	7
Ant: The Language of Claude	10
Talk About Claude Like Humans	15
Old AI = Smart Chatbot	15
Agentic AI = Smart Employee.....	16
Where Claude Fits.....	16
Why OAuth and Entra ID Matter	17
Microsoft Graph = The Nervous System	18
Your "Spider/Crawl on MCP Steroids" Observation	19
What	19
Where	19
How	19
Sequence	20
Output.....	20
What Would a User Actually Do?	21
Why This Is Different From Copilot Chat.....	22
The Big Mental Model.....	23
Connector, MCP, Claude, Foundry.....	24
Tiny Example	26
Tiny Foundry Version	27
Code, Syntax, Text: First UPL	30
Without MCP.....	39
With MCP	39
Minimal Flow	39
Tiny Example	40

Where Entra ID Fits.....	40
Tiny Foundry Version	41
Super Skinny Pseudo-Code	42
The Mental Picture.....	42
Resources.....	43

Bird's-Eye Skinny

Here's the bird's-eye skinny on the Claude Certified Developer – Foundations (CCDV-F) certification.

This is Anthropic's official entry-level technical cert for engineers who actually *build* with Claude (not just chat with it). It's part of their Partner Network ecosystem on Skilljar and targets people writing production code, integrating the API, building agents/workflows, and shipping reliable stuff.

Quick Specs

- **Price:** \$125 USD
- **Audience:** Engineers building with the **Claude API**, **Claude Code**, and **Model Context Protocol (MCP)**
- **Focus:** From first integration all the way to production agents
- **Format:** Exam (similar certs in the program are ~60 questions, proctored, ~2 hours)
- **Prep path:** Dedicated Skilljar learning path with several modules (some free public courses on anthropic.skilljar.com, others more partner-oriented)

The #ccarf-prep anchor on your link is likely a mix-up or cross-reference (CCARF/CCAR-F is the *Architect* cert). This page is for the **Developer** track.

What It Actually Covers (Nuts & Bolts from Prep Objectives)

The prep path breaks it into practical engineering topics:

1. **Model Fundamentals (MSO Foundations)**
Tokens, context window as a "fixed budget," sampling/non-determinism, model tiers, SDK vs. REST.
2. **Production-Grade Prompting, Agents & Tool Use** (biggest hands-on chunk)
System prompts, XML structuring, few-shot, constraints.
Tool schemas + tool-use loops.
Context management in multi-turn/agent scenarios.
Building real agent loops.
3. **Claude Code, MCP & Integration**
Running Claude Code with permission models.
Durable project context.
Packaging workflows as Skills/plugins.
MCP (Model Context Protocol) for connecting to enterprise systems safely without leaking credentials.
This is the "make it configurable and shareable" part.
4. **Production Engineering, Evals & Security**
Building eval suites (define "done," test cases, tracing).

Managing cost/latency/reliability budgets.

Security: prompt injection defense, untrusted input, secret handling.

5. Accelerators & IP Contribution

Packaging reusable components, choosing deployment platforms, contributing back to shared stuff.

Core free/prep course most people start with: “Building with the Claude API” on anthropic.skilljar.com (covers API basics → streaming → structured output → tool use → RAG → agents/workflows → MCP → Claude Code/computer use features). It’s very practical and maps directly to the cert.

Does Everything Run as a 3rd-Party Service?

Yes — 100%.

Claude itself is Anthropic’s hosted model. You never run the weights locally in this certification path. Everything flows through:

- Anthropic’s API (REST or official SDKs — Python, TypeScript, etc.)
- Your code orchestrates the calls, manages conversation history, tools, evals, etc.

You control the *application layer* (your backend, frontend logic, state, UI). Anthropic handles the heavy inference, safety layers, etc.

This is standard for almost all frontier model usage right now.

Takeaways Specifically for Mobile Apps

The cert isn’t mobile-specific, but the patterns transfer directly and are very relevant:

- **Never put your Anthropic API key in the mobile client.** Always proxy through your own backend (serverless function, API gateway, etc.). This is a core production/security takeaway.
- Use **streaming responses** for snappy UX in chat/agent features.
- Implement **tool calling** / agent loops on the backend so the mobile app can trigger complex workflows without exposing everything.
- Context management and prompt caching become important for cost and latency on mobile backends.
- MCP is useful if you want Claude to safely interact with your existing backend services/databases without custom glue code everywhere.
- Security modules (prompt injection, input sanitization) are directly applicable if users can input free text that flows to Claude.
- Eval/tracing practices help you maintain quality as you iterate on mobile-specific agent features.

Bottom line for mobile devs: You'll get solid patterns for "Claude as a smart backend service" rather than trying to do everything client-side. The cert pushes you toward reliable, observable, secure integrations instead of brittle prompt hacks.

Is It "Pretty Light"?

Compared to a full LLM fine-tuning or deep systems course, **yes** — it's deliberately "Foundations." It's not teaching you to train models or write low-level inference code. It's teaching you how to *engineer* reliable applications *around* Claude.

That said, the production focus (evals, budgets, security, MCP integrations) makes it more substantial than pure prompting tutorials. If you've been doing ad-hoc Claude API calls, this will level you up on the engineering discipline side. It's practical and targeted rather than exhaustive.

Instructor / Practitioner POV

From the learning objectives and course structure, the tone is very "ship production systems" oriented:

- Treat the model like a flaky but powerful junior engineer — you need guardrails, evals, observability, and clear interfaces (tools/MCP).
- Focus on **control, cost, and reliability** from day one.
- MCP and tool schemas are highlighted as ways to make integrations cleaner and safer.
- Packaging accelerators/Skills shows they want reusable, shareable patterns rather than one-off scripts.

It's closer to a "best practices for production AI engineering with Claude" than a beginner "hello world API" course.

Recommendation

If you're already building (or want to build) real apps/agents with Claude and want a credential that signals you understand production realities, this is solid. The prep path + free "Building with the Claude API" course will give you the meat even if you don't sit the exam.

Want me to pull more specifics on any module (e.g., tool use patterns, MCP details, eval strategies) or compare it to the Architect track? Just say the word.

"USB-C port for AI" — Model Context Protocol (MCP)

Model Context Protocol (MCP) is an open-source standard introduced by Anthropic in November 2024 for securely connecting AI applications (like Claude) to external data sources, tools, and workflows.

It acts like a "**USB-C port for AI**"—a universal, standardized way to plug AI models into real-world systems without custom one-off integrations for every tool or database.

Why MCP Exists and Why It Matters

Traditional AI integrations require bespoke code for each data source or tool (e.g., separate connectors for Google Drive, Slack, GitHub, databases). This creates fragmentation, duplication, and scaling headaches. MCP solves this by providing a single protocol that both AI clients and servers can implement once, enabling broad interoperability.

Key benefits:

- **For developers:** Faster building and integration; write once, works across many AI apps.
- **For AI apps/agents:** Access to a growing ecosystem of tools, data, and prompts for more capable, context-aware behavior.
- **For users:** More useful agents that can read real data, take actions (with approvals), and maintain context across systems.

It powers features like Claude Code interacting with your local files, enterprise tools, or remote services. Many platforms now support it, including Claude, ChatGPT, VS Code, Cursor, and others.

Core Architecture

MCP uses a **client-server model** with two layers:

- **Data Layer**(JSON-RPC 2.0 based): Handles the actual protocol for exchanging context and actions. It's stateful with lifecycle management (initialization, capability negotiation, shutdown).
- **Transport Layer:** Manages communication channels.
 - **STDIO:** For local servers (fast, no network; common for desktop tools like Claude Desktop).
 - **Streamable HTTP:** For remote servers (supports auth like OAuth, bearer tokens; scalable for production).

Participants:

- **MCP Host:** The AI application (e.g., Claude Desktop or Code).
- **MCP Client:** Component inside the host that connects to servers.
- **MCP Server:** Provides context/tools (can be local or remote).

Core Primitives (The Building Blocks)

MCP servers expose three main types of capabilities that AI models can discover and use dynamically:

1. **Tools** — Executable functions the AI can call (e.g., query a database, fetch weather, control an app). These require user approval for actions. Servers define schemas; AI calls them via `tools/call`.
2. **Resources** — Readable data/context (e.g., file contents, DB records, API responses). Like "file-like" sources the AI can pull from (`resources/list`, `resources/read`).

3. **Prompts** — Reusable templates or workflows (e.g., specialized system prompts, few-shot examples) to guide interactions.

Servers also support discovery (`*/list methods`), and there are client-side primitives like **sampling** (request LLM completions), **elicitation** (ask user for input/confirmation), and **logging**.

Additional features include notifications (for real-time updates, e.g., tool list changes) and experimental tasks for long-running operations.

How It Works in Practice (e.g., with Claude)

1. Connect a server (local via config file or remote via UI).
2. The AI host (Claude) discovers available tools/resources/prompts.
3. During reasoning, Claude can list/call tools, read resources, etc., in a secure, controlled loop.
4. For production/security: Servers handle auth; hosts manage approvals; no credential leaking in many designs.

Example: A weather MCP server exposes `get_alerts(state)` and `get_forecast(lat, lon)` tools. Claude can call them dynamically without you hardcoding API logic.

In the Claude Certified Developer cert context, MCP is emphasized for **safe enterprise integrations**, packaging workflows as shareable connectors, and building robust agents without custom tool schemas for every system.

Getting Started / Building

- **Official site:** modelcontextprotocol.io — specs, SDKs (Python, TS, etc.), quickstarts.
- Anthropic provides courses (e.g., "Introduction to Model Context Protocol") and pre-built servers (GitHub, Postgres, etc.).
- Build a server: Use SDKs with decorators/type hints for tools; run locally or deploy remotely.
- It's open-source and community-driven (GitHub repos for spec, servers, inspector tool).

MCP is still evolving (protocol versions like 2025-06-18), but it's rapidly becoming a de facto standard for agentic AI. In the cert prep, it ties directly into production engineering: reliable tool use, security, and scalable context without reinventing integrations.

If you want code examples, a specific primitive deep-dive, how to build a server, or comparison to traditional tool calling, let me know!

Couple of clarifying Q's:

Model Context Protocol (MCP)

- is an open-source USB c like common standard for securely connecting AI applications (like Claude) to external devices

-- where does oauth fit?

Is Data Layer (JSON-RPC 2.0 based)

- akin to CosmosDB or NoSQL ?

Is reasoning the next generation nomenclature for search albeit with a much better long ball?

- is it Reason Able to infer resources/list might loop over Resources using Tools to reason a return

A weather MCP server exposes get_alerts(state) and get_forecast(lat, lon) tools.

- Claude can call them dynamically without you hardcoding API logic

API logic as

- CRUD over URLs if http?

hardcoding is left for what?

- are client-side primitives aka code run from a prompt so to speak with a new name

what is the "hello-world" simple syntax for a */list method

Answer Grok

Yes, MCP is an open-source, standardized "USB-C for AI" protocol for securely connecting AI applications (e.g., Claude) to external data, tools, and systems.

Where Does OAuth Fit? OAuth 2.1 (with PKCE for public clients) is the recommended/required authorization mechanism primarily for remote (HTTP-based) MCP servers. It handles secure authentication and delegated access without hardcoding credentials.

Local/STUDIO servers (running on the same machine, e.g., Claude Desktop) often rely on OS-level permissions or simpler configs—no full OAuth needed.

- Remote servers use OAuth flows (Authorization Code with PKCE is common). The MCP client acts as an OAuth client, the server as a resource server, and there's typically an authorization server (e.g., your IdP) that issues tokens.
- This keeps things secure: user consent, token-based access, dynamic client registration, etc. It integrates with existing enterprise identity systems.

OAuth lives at the transport layer for remote connections, enabling safe production use while the data layer (JSON-RPC) handles the actual tool/resource exchanges. Is the Data Layer (JSON-RPC 2.0) Akin to CosmosDB or NoSQL? No—not really.

- JSON-RPC 2.0 is a lightweight remote procedure call (RPC) protocol. It defines a simple request-response (and notification) format over JSON for clients and servers to invoke methods (e.g., tools/list, tools/call, resources/read).

- It's not a database at all. Think of it more like a standardized API contract for method calls, similar to gRPC or older XML-RPC, but simpler and JSON-based.
- CosmosDB/NoSQL are actual data stores. MCP's data layer just structures communication about context (tools/resources/prompts); the underlying data can live anywhere (files, DBs, APIs).

Is "Reasoning" the Next-Gen Nomenclature for Search (with Better Long Ball)? Kind of, but broader. Reasoning in LLMs refers to step-by-step thinking, planning, tool use, iteration, and inference (e.g., chain-of-thought, agents). It includes advanced search/retrieval (like RAG or tool-augmented search) but goes much further: multi-step planning, evaluation, code execution, decision-making, and handling uncertainty. The "long ball" comes from better context management, tool calling, and loops—allowing the model to "search" its environment dynamically and reason over results, not just keyword match or vector similarity. Can Reasoning Infer resources/list Might Loop Over Resources Using Tools? Yes, absolutely. That's a core strength of agentic setups with MCP:

- The LLM (via the client) can discover available resources/tools (* /list methods).
- It can then use tools to reason, filter, or process them (e.g., call a tool that queries a resource, iterate based on results, or chain actions).
- Reasoning loops allow dynamic exploration: list resources → read promising ones → use tools on the data → refine. MCP's primitives + notifications support this without hardcoding.

Weather MCP Server Example: Dynamic Calls Without Hardcoding API Logic—Yes—the AI (Claude) discovers and calls `get_alerts(state)` or `get_forecast(lat, lon)` at runtime via the protocol. You (or the server author) implement the backend logic once in the MCP server; the model doesn't need embedded URLs, auth, or call patterns. API logic as CRUD over URLs if HTTP?

Often yes for remote tools (the server handles HTTP calls to weather APIs, DBs, etc.). The model just sees abstract tools/resources. Hardcoding is left for what?

- Implementing the actual business logic inside the MCP server (e.g., the Python/TS code that hits the NWS API, formats results, handles errors).
- Server-side auth, rate limiting, permissions, data transformation.
- The client/host still manages high-level orchestration, approvals, and LLM prompting.

This shifts from "paste API code into prompts" to reusable, discoverable, secure components. Client-Side Primitives (Sampling, Elicitation, etc.) These are not exactly "code run from a prompt," but they let the MCP server request capabilities from the client/host during a session:

- Sampling (sampling/createMessage): Server asks the client to run an LLM completion (useful for nested reasoning without the server embedding its own LLM).
- Elicitation: Server asks the user (via client) for input/confirmation (structured forms, yes/no, etc.).
- Logging: Server sends debug/info to the client.

They're protocol methods for richer, interactive agent flows—server-driven but client-mediated for security/control.

"Hello World" Simple Syntax for a */list MethodHere's a basic example of a tools/list request (JSON-RPC) from client to server:

```
json
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {
    // Optional cursor for pagination
    "cursor": null
  }
}
```

A typical server response might include an array of tool definitions (name, description, input schema). Similar for resources/list or prompts/list. SDKs abstract this heavily—you define tools with decorators/type hints, and */list is auto-handled

Ant: The Language of Claude

"Ant" refers to Anthropic's official CLI (command-line tool) for the Claude platform. It is not a programming language, but rather a tool that allows developers to manage Claude APIs, managed agents, and sessions directly from the terminal. [1, 2, 3]

Ant and MCP (Model Context Protocol) serve different, complementary roles in the Claude ecosystem: [1, 2, 3, 4]

- **Ant CLI:** A command-line tool written in Go that maps Claude's API endpoints into terminal commands (e.g., sending messages or deploying agents). [1, 2, 3]
- **MCP:** An open standard created by Anthropic that acts like a USB port for AI, allowing Claude to securely connect to external tools, data sources, and service

You are spot on: **ant is a command-line interface (CLI) tool, whereas Go is a full programming language.** [1]

When looking at the documentation on the Anthropic homepage, the examples illustrate two entirely different ways of interacting with Claude:

- **The Go example** is full backend source code utilizing Anthropic's Go SDK. Because Go is a statically typed, compiled programming language, it requires boilerplate setup like package declarations, importing libraries, defining a main() function, and explicit error handling. This makes it look quite wordy. [2, 3, 4]
- **The ant example** is a terminal command. Since ant is a pre-compiled binary tool, it abstracts away all that boilerplate. You do not need imports or syntax declarations; you just pass text flags or files right from your shell. [1, 2, 5]

The "Go and Git" Connection

To clarify your thought on Go being "familiar with Git," Go does have a tight-knit architectural relationship with Git, but not in the way it executes code:

- **Go Dependency Management:** Go uniquely uses Git URLs directly in its code to manage packages (e.g., import "://github.com"). When you build a Go program, it uses Git behind the scenes to fetch those repositories.
- **The ant Tool Backbone:** The ant CLI tool itself is actually *written* in Go. Anthropic's engineers wrote the wordy Go code, compiled it into a lightweight binary, and distributed it so that you only have to type short commands like ant messages create. [2, 6, 7]

Comparison Table

Aspect	Go SDK Example	ant CLI Example
What it is	A backend programming language.	A pre-compiled terminal tool.
Boilerplate	High (Requires imports, functions, types).	None (Single-line shell commands).
Best Used For	Building complex apps, web servers, or backend pipelines.	Quick scripts, cron jobs, CI/CD testing, and quick terminal queries.

If you want to quickly test a prompt without dealing with Go compilation, the [anthropics/anthropic-cli](#) repository on GitHub outlines how you can install ant using Homebrew (brew install anthropics/tap/ant) or via Go itself (go install). [2, 8]

Would you like help **setting up the ant CLI** with your API key, or would you prefer a quick template for writing a **lightweight automation script**?

Skilljar

General Product Comparison, why different from each other as a service

Claude on web, Claude Code, Claude Console, Claude Chat, Claude CoWork, Claude API

Here's a clear product comparison of the main Claude surfaces from Anthropic (as of mid-2026). They share the same underlying models but differ sharply in interface, autonomy, file/system access, target user, and billing. Quick Mental Model One "brain" (Claude models), multiple specialized "hands and environments":

Product	Primary Interface	Core Job	Autonomy Level	Best For	Typical Access / Billing
Claude on web / Claude Chat (claude.ai)	Browser + mobile chat	Conversational thinking & drafting	Low (turn-by-turn)	Research, writing, analysis, brainstorming	Free / Pro (\$20) / Max / Team / Enterprise subscription
Claude Code	Terminal, IDE (VS Code/JetBrains), desktop, web, mobile	Agentic software engineering	High (plans + executes)	Coding, debugging, refactors, git, shipping	Pro+ subscription or Console/API credits / cloud providers
Claude Console	Web developer portal (console.anthropic.com / platform.claude.com)	Prompt testing, evals, key management, Workbench	N/A (dev tooling)	Building & iterating on API apps	API account (pay-per-token credits)
Claude CoWork (Cowork)	Desktop app (primary), web & mobile beta	Agentic knowledge work	High (goal → finished deliverable)	Docs, research briefs, file organization, multi-app tasks	Pro / Max / Team / Enterprise (higher usage burn)
Claude API	Programmatic (HTTP + SDKs)	Embed Claude in your products	You control the loop	Production apps, agents, automation, high volume	Pure pay-per-token (separate from chat subscriptions)

Why They Differ as Services1. Claude on web / Claude Chat (claude.ai)

- Pure conversational interface. You prompt → Claude replies → you iterate.
- Strong features: Projects (persistent context), Artifacts (live previews/code/charts), web search, file uploads.
- No direct local file system or shell access in plain chat.

- Designed for knowledge workers who want a thinking partner, not an autonomous worker.
- Easiest on-ramp; zero setup.

2. Claude Code

- Agentic coding agent. You give a goal; it explores the codebase, plans, edits files, runs commands/tests, manages git, and shows diffs for approval.
- Runs where developers live (CLI, IDE extensions, desktop). Supports MCP for tools, computer use, long-running tasks, subagents.
- Same agentic DNA as CoWork but specialized for software engineering.
- Can also be used via Claude.ai web for lighter coding sessions.
- Requires Pro+ plan (or Console/API billing). Free tier has no/limited access.

3. Claude Console

- Developer control plane, not an end-user chat product.
- Create API keys, manage billing/usage, Workbench for prompt engineering, evaluation tools, team collaboration on prompts, analytics.
- Where you experiment before wiring the API into production code.
- Completely separate account/billing from claude.ai subscriptions.
- Think of it as the “ops & experiment dashboard” for the API.

4. Claude CoWork (Cowork)

- “Claude Code for non-coders.” Same agentic approach (plan → act → iterate), but aimed at knowledge work.
- You grant folder access; Claude reads/edits/creates local files, connects to apps (via connectors/MCP), produces finished deliverables (reports, organized folders, drafts).
- Desktop-first (deepest file + browser access); web/mobile beta for starting/monitoring tasks that can continue in the cloud.
- Explicitly differentiated from Chat (Chat can’t act on files) and from Claude Code (Code is for software).
- Consumes usage faster than plain Chat because of longer agent runs.

5. Claude API

- The raw model access layer. You (or your app) send messages, manage tools/context, handle streaming, evals, etc.

- Full control: custom agents, production integrations, mobile backends, high-volume pipelines, MCP servers/clients.
- No built-in UI, file system, or chat history—you build those.
- Billed purely by tokens (input/output/caching). Completely independent of Pro/Max chat subscriptions.
- Powered by the same models, but you own the orchestration, security, and cost controls.

Key Distinctions That Drive the Differences

- Interface & environment — Chat is browser-native conversation; Code lives in the developer toolchain; CoWork lives on the desktop file system; Console is the admin/dev portal; API is headless infrastructure.
- Autonomy vs. conversation — Chat keeps you in the loop every turn. Code and CoWork take a goal and execute multi-step work (with visibility and approvals).
- Access model — Chat/CoWork/Code (subscription) give managed experiences with usage limits. API + Console give raw, metered, programmable access.
- Target user — Chat/CoWork = broad knowledge workers. Code = software engineers. Console/API = builders embedding Claude.
- Billing isolation — Chat subscription ≠ API credits. Paying for Pro does not give you API tokens, and vice versa (though Code can draw from either in some setups).

Practical recommendation

- Everyday thinking/writing → Claude Chat (web).
- Building or shipping software → Claude Code.
- Delegating multi-step office/research work → Claude CoWork.
- Integrating Claude into your own apps or automations → Claude API (+ Console for development).

Most teams end up using more than one. The products are intentionally specialized so the same model can feel natural in very different workflows.

Talk About Claude Like Humans

- Hey Co - Claude Talk - bringing connecting to sources with oauth as a service into the mix -
- & Microsoft Graph permissions via Entra ID use in Microsoft cloud environments
- <https://azure.microsoft.com/en-us/blog/claude-in-microsoft-foundry-is-now-generally-available/>
- <https://azure.microsoft.com/en-us/products/ai-foundry/agent-service/>

Developers can access Claude through the Messages API and use

- prompt caching
- extended thinking
- tool streaming

Foundry Agent Service

- reasoning core to orchestrate multi-step
- planning
- tool use
- task execution

--- across enterprise systems

And now in Micro Talk - is this using a prompt to describe an informational/media goal

- & the Spider or Crawl (on MCP Steriods)

Know How, What, Where to look or "Reasons" to assemble the final package so to speak for the user?

for example in Off365

- What would a user do

-- Open a prompt from a downloaded Claude local install

connect to... and do...

Expecting ? as the result?

Large Loose Broad Strokes for a Mental Picture of Why I Need Claudes Help

Co Sez: Tim, the way I'd build the mental picture is this:

Old AI = Smart Chatbot

You ask:

"Summarize my sales pipeline."

The model responds from what you pasted into the chat.

It knows language. It does **not** know your environment.

Agentic AI = Smart Employee

Now you ask:

"Prepare next week's executive sales review."

The agent:

1. Figures out what information is needed.
2. Identifies where that information lives.
3. Connects to those systems.
4. Retrieves data.
5. Analyzes it.
6. Creates the deliverables.

That is the shift Microsoft Foundry Agent Service is targeting. Microsoft describes Foundry Agent Service as using models such as Claude as a reasoning core for multi-step planning, tool use, and task execution across enterprise systems. [\[azure.microsoft.com\]](https://azure.microsoft.com), [\[devblogs.microsoft.com\]](https://devblogs.microsoft.com)

Where Claude Fits

Claude is basically the **reasoning engine**.

Microsoft Foundry provides:

- Identity
- Security
- Governance
- Networking
- Billing
- Agent framework

Claude provides:

- Reasoning
- Planning
- Tool selection

- Complex decision making

Microsoft's announcement specifically notes Claude can be accessed through the Messages API and supports prompt caching, extended thinking, and tool streaming. azure.microsoft.com, claude.com

Think:

User

↓

Foundry Agent Service

↓

Claude

↓

Tools / Connectors / MCP / APIs

↓

- Microsoft 365
- SharePoint
- Teams
- Graph
- Salesforce
- SAP
- Databases
- Web

Why OAuth and Entra ID Matter

Without OAuth:

Agent = blind

With OAuth:

Agent can act as you

When you sign in through Microsoft Entra ID:

User

- → grants permission
- → agent receives token
- → Graph API access

Now Claude doesn't need copies of data.

Instead:

Claude reasons

"To answer this request

I need Teams data"

↓

Calls Graph

↓

Gets Teams data

↓

Analyzes

↓

Produces result

This is much more secure.

Microsoft Graph = The Nervous System

Think of Microsoft Graph as:

Universal Microsoft 365 API

It can expose:

- Outlook
- Teams
- OneDrive

- SharePoint
- Planner
- To Do
- Users
- Groups
- Calendar
- Files

through a common interface.

So if Claude has Graph permissions:

- Read Mail
- Read Calendars
- Read Teams chats
- Read Files
- Read SharePoint lists

it has access to the information needed for many enterprise tasks.

Your "Spider/Crawl on MCP Steroids" Observation

Honestly...

that's not a bad intuition.

The difference is that a traditional crawler simply gathers information.

An agent reasons about:

What

What information is required?

Where

Where is it likely stored?

How

How do I get authorized access?

Sequence

What order should tasks execute?

Output

What final artifact should be produced?

So instead of:

Crawl everything

it's more like:

Goal driven discovery

Example:

"Build a project status package for leadership."

Agent reasoning:

Need project plans

→ Planner

Need meeting notes

→ Teams

Need status deck

→ SharePoint

Need risks

→ Jira

Need milestones

→ Project Online

Assemble report

→ PowerPoint

That's far beyond traditional crawling.

What Would a User Actually Do?

Let's say Claude Desktop eventually has Microsoft connectors configured.

A user might simply type:

Create a quarterly business review from all customer-facing Teams meetings during the last 90 days.

The agent may:

Authenticate via Entra ID

↓

Search Teams recordings

↓

Find transcripts

↓

Summarize themes

↓

Pull CRM data

↓

Compare against revenue

↓

Generate PowerPoint

↓

Save to SharePoint

↓

Notify Team

User gave:

Goal

Agent figured out:

Process

Why This Is Different From Copilot Chat

Traditional Copilot:

Question

→ Answer

Agentic Foundry:

Goal

→ Plan

→ Tool execution

→ Data retrieval

→ Iteration

→ Deliverable

The deliverable might be:

- Word document
 - PowerPoint
 - Excel workbook
 - SharePoint page
 - Email draft
 - Teams post
 - Workflow completion
-

The Big Mental Model

Think of these layers:

Claude

=

Brain

Foundry Agent Service

=

Project Manager

MCP / Connectors / APIs

=

Hands and Eyes

Entra ID

=

Badge and Security Clearance

Microsoft Graph

=

Enterprise Information Highway

Microsoft 365

=

The Actual Company Data

So when you ask:

"Why would I need Claude's help?"

The answer is:

Because Claude is no longer just generating text.

In the Foundry + Agent Service model, Claude becomes the reasoning layer that can determine **what information is needed, where it lives, how to access it via Entra/OAuth, what tools to invoke,**

and how to assemble a finished business outcome from many enterprise systems.

[\[azure.microsoft.com\]](https://azure.microsoft.com), [\[devblogs.microsoft.com\]](https://devblogs.microsoft.com), [\[learn.microsoft.com\]](https://learn.microsoft.com)

That's the leap from *chatbot* to *digital knowledge worker*.

Connector, MCP, Claude, Foundry

Absolutely. Let's make MCP as boring and practical as possible.

A lot of MCP discussions sound like:

"AI agents dynamically orchestrating heterogeneous tool ecosystems..."

But the skinniest real-world example is much simpler.

Without MCP

Your app needs SharePoint files.

You write:

- 1 App
- 2 └─ OAuth login
- 3 └─ Graph token handling
- 4 └─ Refresh tokens
- 5 └─ Graph API calls
- 6 └─ Error handling
- 7 └─ SharePoint logic
- 8 └─ File retrieval

Lots of plumbing.

With MCP

Your app says:

- 1 "Get file QBR.pptx from SharePoint"

The MCP server handles:

- 1 OAuth
- 2 Connection
- 3 Permissions
- 4 API calls
- 5 Responses

The app only focuses on business logic.

Minimal Flow

- 1 User
- 2 ↓
- 3 App
- 4 ↓
- 5 MCP Client
- 6 ↓
- 7 SharePoint MCP Server
- 8 ↓
- 9 Microsoft Graph
- 10 ↓
- 11 SharePoint

Response comes back:

- 1 File
- 2 Metadata
- 3 Content

Then:

- 1 Connection ends

Done.

Tiny Example

Imagine a sales dashboard.

User clicks:

```
1 Show latest forecast
```

App code:

```
1 forecast = mcp.call_tool(  
2     "sharepoint_get_file",  
3     path="/Sales/Forecast.xlsx"  
4 )  
5  
6 render_forecast(forecast)
```

The app doesn't know:

- Graph
- SharePoint APIs
- OAuth token flow

The MCP server does.

Where Entra ID Fits

The user logs in once:

```
1 Login → Entra ID
```

Entra issues token.

MCP server receives token.

MCP server can now call:

```
1 Microsoft Graph
```

as the user.

So:

```
1 Tim  
2 ↓
```

3	App
4	↓
5	MCP
6	↓
7	Graph
8	↓
9	SharePoint

The permissions are still enforced.

If Tim can't see the file:

1	Access Denied
---	---------------

MCP doesn't magically bypass security.

Tiny Foundry Version

Now insert Claude or GPT.

User types:

Summarize last month's project status.

Flow:

1	User Prompt
2	↓
3	Foundry Agent
4	↓
5	Claude reasoning

Claude decides:

1	Need project reports
---	----------------------

Calls MCP tool:

1	sharepoint_search_reports()
---	-----------------------------

Gets documents.

Then:

```
1  summarize()
```

Returns:

3 projects on track 1 delayed Budget variance 4% Top risk is vendor delivery

The model never directly knew SharePoint.

It simply knew:

```
1  A tool exists
2  Use it
```

Super Skinny Pseudo-Code

```
1  # User already signed in via Entra
2
3  question = "Summarize project status"
4
5  docs = mcp.call_tool(
6      "sharepoint_get_documents",
7      folder="/Projects/Status"
8  )
9
10 answer = foundry_model.chat(
11     prompt=question,
12     context=docs
13 )
14
15 return answer
```

That's honestly about as small as the idea gets.

The Mental Picture

Think of MCP as:

- 1 USB for enterprise data

Before USB:

- 1 Custom cable
- 2 Custom driver
- 3 Custom port

After USB:

- 1 Plug in
- 2 Use device

Before MCP:

- 1 Custom Graph code
- 2 Custom Salesforce code
- 3 Custom Jira code
- 4 Custom SAP code

After MCP:

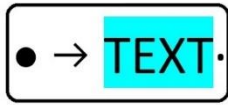
- 1 Tool available
- 2 Call tool
- 3 Get result

And Foundry agents/Claude become the thing smart enough to decide:

"For this question I should use the SharePoint tool, then the Teams tool, then the Outlook tool."

while Entra ID and Graph ensure all access stays inside the permissions the logged-in user already has. That's why Microsoft shops are paying attention: it potentially reduces a lot of integration code while keeping the security model they already trust.

Code, Syntax, Text: First UPL



- → **TEXT**: the first UPL: (Universal Programming Language)
- → DO-C.R.U.D. with Content on the Web 

Tim, I actually think your "**TEXT is the First UPL (Universal Programming Language)**" idea maps surprisingly well to where agents, MCP, Foundry, and modern IDEs appear to be heading.

The image itself appears to show:

- A box with an arrow pointing toward the word **TEXT**
- The statement:
 - **TEXT: the first UPL (Universal Programming Language)**
- A second statement:
 - **DO C.R.U.D. with Content on the Web**
- A small "Td Brand" style logo mark

The central idea I'm seeing is:

1	Intent
2	↓
3	TEXT
4	↓
5	Execution Environment

rather than:

1	Human
2	↓
3	Specific Language
4	↓
5	Specific SDK
6	↓
7	Specific API

Where Does the "Super Skinny Pseudo-Code" Run?

The answer is:

Anywhere that can interpret the TEXT.

The runtime is separate from the intent.

For example:

- 1 "Get project status from SharePoint
- 2 Summarize it
- 3 Create a report"

could execute in:

- Foundry Agent
- Claude Desktop
- VS Code
- Copilot Studio
- Azure Function
- Local Python script
- Power Automate

The TEXT is the request.

The execution layer decides:

- 1 What tools are needed?
- 2 What credentials are needed?
- 3 What APIs are needed?

MCP as an Execute-As Service

I think your phrase:

Giant Execute As Answer/Service

is actually not bad.

Imagine:

1	User
2	↓
3	Intent
4	↓
5	Agent
6	↓
7	Execute-as-user
8	↓
9	Systems

The entire point of:

- Entra ID
- OAuth
- Graph
- MCP

is that the execution occurs with the identity of the user.

So an agent is often doing:

1	Answer =
2	Result of executing actions

rather than:

1	Answer =
2	Text generated from training

Huge difference.

Foundry's Role

In a Microsoft shop I'd draw it this way:

1	Foundry
2	=

3	Agent Host
1	Claude/GPT
2	=
3	Reasoning Engine
1	MCP
2	=
3	Tool Interface
1	Entra ID
2	=
3	Identity
1	Graph
2	=
3	Microsoft Data Access Layer
1	M365
2	=
3	Actual Data

If TEXT Is The UPL

Then the stack becomes interesting.

Instead of:

1	Python
2	C#
3	JavaScript
4	PowerShell
5	SQL

you start with:

1	TEXT
---	------

Example:

- 1 Find all project updates
- 2 from the last 30 days.
- 3
- 4 Summarize risks.
- 5
- 6 Create a PowerPoint.
- 7
- 8 Save in SharePoint.

That's your source.

Now something must convert intent into execution.

The New Compiler

Historically:

- 1 Source Code
- 2 ↓
- 3 Compiler
- 4 ↓
- 5 Machine Code

Your model becomes:

- 1 TEXT
- 2 ↓
- 3 Reasoning Engine
- 4 ↓
- 5 Tool Selection
- 6 ↓
- 7 Execution

Almost:

- 1 Natural Language Compiler

What Parts Exist?

If we minimize tools and maximize automation:

1 1. Editor

Maybe:

- VS Code
- Claude Desktop
- Copilot
- Browser

1 2. Reasoning Layer

Claude/GPT

Reads TEXT.

Determines:

- | |
|---------|
| 1 What |
| 2 Where |
| 3 How |

1 3. Tool Registry

MCP

Provides:

- | |
|-------------------|
| 1 SharePoint Tool |
| 2 Teams Tool |
| 3 SQL Tool |
| 4 Filesystem Tool |
| 5 GitHub Tool |

1 4. Identity Layer

Entra ID

Determines:

1 Can Tim access this?

1 5. Execution Layer

Could be:

- Azure Functions
- Containers
- Agent Service
- Local Machine

This is where actions actually happen.

1 6. Result Layer

Outputs:

- Email
- PPTX
- DOCX
- JSON
- Dashboard
- Web Page

Your Mechanic Analogy

I really like:

Put less tools in the mechanic's hand.

Today's developer:

1 Needs to know

- 2 Graph
- 3 OAuth
- 4 REST
- 5 SDKs
- 6 Languages
- 7 Frameworks

Your future-state vision is:

- 1 Mechanic writes TEXT

Example:

- 1 Monitor the sales folder.
- 2
- 3 When a new spreadsheet arrives:
- 4
- 5 Calculate variance.
- 6
- 7 Email leadership.
- 8
- 9 Store summary.
- 10
- 11 Update dashboard.

The system figures out:

- 1 Power Automate?
- 2 Python?
- 3 Azure Function?
- 4 Graph?
- 5 SQL?

The Extreme Version

The really radical idea is this:

- 1 TEXT
- 2 is not documentation.
- 3 TEXT
- 4 is the application.

Meaning:

- 1 Application
- 2 =
- 3 Intent

not

- 1 Application
- 2 =
- 3 Compiled Code

Foundry Agents, MCP, Claude, Copilot Studio, LangGraph, Semantic Kernel, and similar technologies all feel like early steps toward that direction:

- 1 Human Intent
- 2 ↓
- 3 TEXT
- 4 ↓
- 5 Agent
- 6 ↓
- 7 Tool Selection
- 8 ↓
- 9 Execution
- 10 ↓
- 11 Outcome

In that mental model, VS Code becomes less of a "coding editor" and more of an **intent editor**, while MCP becomes a standardized way for that intent to reach whatever tools, services, and enterprise systems are required to make the intent real.

Absolutely. Let's make MCP as boring and practical as possible.

A lot of MCP discussions sound like:

"AI agents dynamically orchestrating heterogeneous tool ecosystems..."

But the skinniest real-world example is much simpler.

Without MCP

Your app needs SharePoint files.

You write:

```
1 App
2   | OAuth login
3   | Graph token handling
4   | Refresh tokens
5   | Graph API calls
6   | Error handling
7   | SharePoint logic
8   | File retrieval
```

Lots of plumbing.

With MCP

Your app says:

```
1 "Get file QBR.pptx from SharePoint"
```

The MCP server handles:

```
1 OAuth
2 Connection
3 Permissions
4 API calls
5 Responses
```

The app only focuses on business logic.

Minimal Flow

```
1 User
2 ↓
3 App
```

```
4   ↓
5   MCP Client
6   ↓
7   SharePoint MCP Server
8   ↓
9   Microsoft Graph
10  ↓
11  SharePoint
```

Response comes back:

```
1  File
2  Metadata
3  Content
```

Then:

```
1  Connection ends
```

Done.

Tiny Example

Imagine a sales dashboard.

User clicks:

```
1  Show latest forecast
```

App code:

```
1  forecast = mcp.call_tool(
2      "sharepoint_get_file",
3      path="/Sales/Forecast.xlsx"
4  )
5
6  render_forecast(forecast)
```

The app doesn't know:

- Graph
- SharePoint APIs
- OAuth token flow

The MCP server does.

Where Entra ID Fits

The user logs in once:

1 Login → Entra ID

Entra issues token.

MCP server receives token.

MCP server can now call:

1 Microsoft Graph

as the user.

So:

```
1 Tim
2   ↓
3 App
4   ↓
5 MCP
6   ↓
7 Graph
8   ↓
9 SharePoint
```

The permissions are still enforced.

If Tim can't see the file:

1 Access Denied

MCP doesn't magically bypass security.

Tiny Foundry Version

Now insert Claude or GPT.

User types:

Summarize last month's project status.

Flow:

```
1 User Prompt
2   ↓
3 Foundry Agent
4   ↓
5 Claude reasoning
```

Claude decides:

1 Need project reports

Calls MCP tool:

```
1 sharepoint_search_reports()
```

Gets documents.

Then:

```
1 summarize()
```

Returns:

```
3 projects on track 1 delayed Budget variance 4% Top risk is vendor delivery
```

The model never directly knew SharePoint.

It simply knew:

```
1 A tool exists
2 Use it
```

Super Skinny Pseudo-Code

```
1 # User already signed in via Entra
2
3 question = "Summarize project status"
4
5 docs = mcp.call_tool(
6     "sharepoint_get_documents",
7     folder="/Projects/Status"
8 )
9
10 answer = foundry_model.chat(
11     prompt=question,
12     context=docs
13 )
14
15 return answer
```

That's honestly about as small as the idea gets.

The Mental Picture

Think of MCP as:

```
1 USB for enterprise data
```

Before USB:

```
1 Custom cable
2 Custom driver
```

3 Custom port

After USB:

- 1 Plug in

2 Use device

Before MCP:

- 1 Custom Graph code

2 Custom Salesforce code

3 Custom Jira code

4 Custom SAP code

After MCP:

- 1 Tool available

2 Call tool

3 Get result

And Foundry agents/Claude become the thing smart enough to decide:

"For this question I should use the SharePoint tool, then the Teams tool, then the Outlook tool."

while Entra ID and Graph ensure all access stays inside the permissions the logged-in user already has. That's why Microsoft shops are paying attention: it potentially reduces a lot of integration code while keeping the security model they already trust.

Resources

1. [What is the Model Context Protocol \(MCP\)? - Model Context Protocol](#)
- 2.